Algorithm Design Strategies

Applications and Examples

Example 1

An array A[1..n] is ***Unimodal*** if it consists of an increasing sequence followed by a decreasing sequence, or more precisely, if there is an index m ϵ {1,2,..,n} such that

- A[i] < A[i+1] for all 1 <= i < m and
- A[i] > A[i+1] for all m <= i < n

In particular, A[m] is the maximum element, and it is unique “locally maximum” element surrounded by smaller elements. Give an algorithm to compute the maximun element of a ***Unimodal*** input array A[1..n], analyze the algorithm. Prove the correctness of your algorithm and prove the bound on its running time.

Divide and Conquer

Example 2

- Given a set, *P*, of *n* teams in some sport, a *round-robin tournament* is a collection of games in which each team plays each other team exactly once. Such round-robin tournaments are often used as the first round for establishing the order of teams (and their seedings) for later single- or double-elimination tournaments. Design an efficient algorithm for constructing a round-robin tournament for a set, *P*, of *n* teams assuming *n* is a power of 2.

Divide and Conquer

Example 3

- Eggs break when dropped from great enough height. Specifically, there must be a floor f in any sufficiently tall building such that an egg dropped from the fth floor breaks, but one dropped from the $(f - 1)$st floor will not. If the egg always breaks, then $f = 1$. If the egg never breaks, then $f = n + 1$. You seek to find the critical floor f using an n-story building. The only operation you can perform is to drop an egg off some floor and see what happens. You start out with k eggs, and seek to drop eggs as few times as possible. Broken eggs cannot be reused. Let $E(k, n)$ be the minimum number of egg droppings that will always suffice.

Dynamic Programming

Example 4

- In the *single-processor scheduling problem*, we are given a set of *n* jobs *J*. Each job *i* has a processing time *ti*, and a deadline *di*. A feasible schedule is a permutation of the jobs such that when the jobs are performed in that order, every job is finished before its deadline. The greedy algorithm for single-processor scheduling selects the job with the earliest deadline first. Show that if a feasible schedule exists, then the schedule produced by this greedy algorithm is feasible

Greedy Algorithm

Example 5

- Consider the problem of storing *n* books on shelves in a library. The order of the books is fixed by the cataloging system and so cannot be rearranged. Therefore, we can speak of a book *bi*, where 1 ≤ *i* ≤ *n*, that has a thickness *ti* and height *hi*. The length of each bookshelf at this library is *L*. Suppose all the books have the same height *h* (i.e. , *h* = *hi* = *hj* for all *i*, *j*) and the shelves are all separated by a distance of greater than *h*, so any book fits on any shelf. The greedy algorithm would fill the first shelf with as many books as we can until we get the smallest *i* such that *bi* does not fit, and then repeat with subsequent shelves. Show that the greedy algorithm always finds the optimal shelf placement, and analyze its time complexity

Greedy Algorithm

Example 6

- Amrita is conducting its annual book exhibition. There are n different books on sale in the exhibition. The i^{th} book in the exhibition is priced at Rs. Pi. John is a first year student (Reg. No. BL.EN.U4CSE19020). John plans to spend Rs. W to buy books during the exhibition. He wants to impress upon his parents and buy as many of books as possible. However, he does not want to buy two copies of the same book.

a) Suggest an algorithm which John can use to buy as many books as possible with Rs. W.

b) What is the complexity of your algorithm?

Dynamic Programming

Example 7

- You are playing a game where there is a monster is chasing you in a maze. The maze has a graph structure, possibly with cycles. Your goal is to escape from the maze before the monster catches you and eats you. You know the following: the maze layout, your location in the maze, and the monster's location. However, the escape door is invisible unless you are right in front of it. You and the monster alternate moves. At each move, the player moving (you or the monster) must move to a node adjacent to the mover's current position in the maze. After getting eaten multiple times, you decide to write a program to figure out your moves. That is, at each step in the game, you will consult your program to decide where to move next. Your strategy won't necessarily guarantee that you escape, but it should greatly improve your chances. Discuss the issues/challenges of this problem, discuss the graph algorithms you might use and why, your general strategy if you have one, and any other details you feel are important. Note, this problem is intentionally left vague. Be creative and thoughtful.

Backtracking